



Techniques of High-Performance Computing - PHAS0102

Week 4: Sparse Linear Algebra

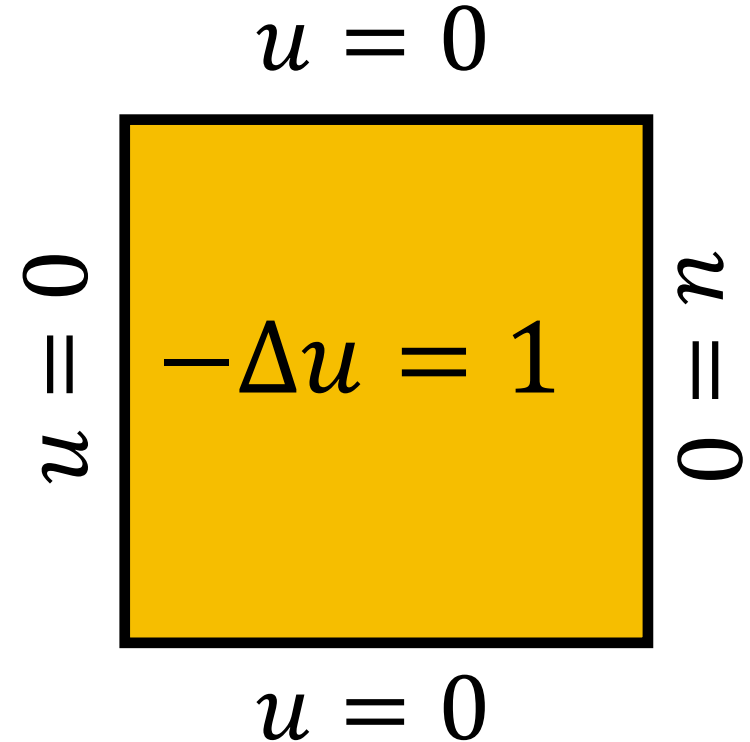
Example 2D problem

$$-\Delta u = 1$$

$$\Delta u = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}$$

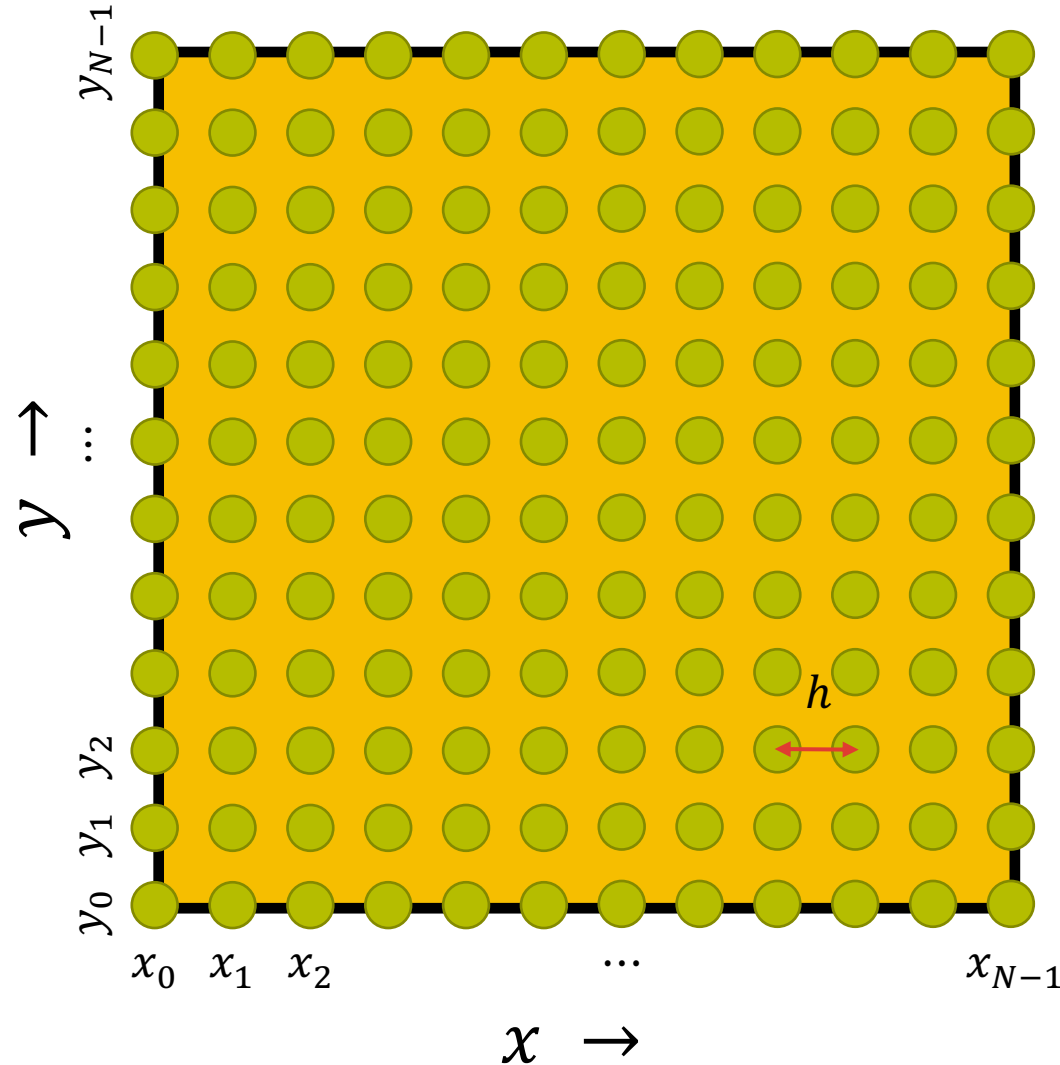
$$\Omega = [0, 1]^2$$

$$u = 0 \text{ on the boundary}$$



How would we solve it?

Discretization



$$x_i = ih$$

$$y_i = ih$$

For N evenly spaced
grid points:

$$h = \frac{1}{N - 1}$$

Discretization

$$u \rightarrow u_{i,j}$$

$$u_{i,j} = u(x_i, y_j)$$

Derivatives

- How do we handle derivatives?
- *Finite Differences Method*

$$\frac{\partial u}{\partial x} \approx \frac{u_{i+1,j} - u_{i,j}}{h}$$

$$\frac{\partial^2 u}{\partial x^2} \approx \frac{u_{i-1,j} - 2u_{i,j} + u_{i+1,j}}{h^2}$$

$$\frac{\partial u}{\partial y} \approx \frac{u_{i,j+1} - u_{i,j}}{h}$$

$$\frac{\partial^2 u}{\partial y^2} \approx \frac{u_{i,j-1} - 2u_{i,j} + u_{i,j+1}}{h^2}$$

Finite Differences

$$\Delta u = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \approx \frac{u_{i-1,j} - u_{i,j-1} - 4u_{i,j} + u_{i+1,j} + u_{i,j+1}}{h^2}$$

$$-\Delta u = 1$$

$$\frac{4u_{i,j} - u_{i+1,j} - u_{i,j+1} - u_{i-1,j} - u_{i,j-1}}{h^2} = 1$$

$$h = \frac{1}{N-1}$$

$$(N-1)^2 (4u_{i,j} - u_{i+1,j} - u_{i,j+1} - u_{i-1,j} - u_{i,j-1}) = 1$$

For $0 < i, j < N-1$

$u_{i,j} = 0$ on the boundary

Poisson Problem

- N grid points for x and y
- $u_{0,0}, u_{1,0}, u_{2,0} \dots, u_{N-1,0} \dots u_{N-1,N-1}$
- N^2 variables

Matrices

$$4a_0 + 3a_1 = 2$$

$$a_0 - a_3 = 1$$

$$-a_2 - a_3 = 0$$

$$2a_0 = 1$$

$$4a_0 + 3a_1 + 0a_2 + 0a_3 = 2$$

$$1a_0 + 0a_1 + 0a_2 - 1a_3 = 1$$

$$0a_0 + 0a_1 - 1a_2 - 1a_3 = 0$$

$$2a_0 + 0a_1 + 0a_2 + 0a_3 = 1$$

$$4a_0 + 3a_1 + 0a_2 + 0a_3 = 2$$

$$1a_0 + 0a_1 + 0a_2 - 1a_3 = 1$$

$$0a_0 + 0a_1 - 1a_2 - 1a_3 = 0$$

$$2a_0 + 0a_1 + 0a_2 + 0a_3 = 1$$

$$Ax = b$$

$$\begin{bmatrix} 4 & 3 & 0 & 0 \\ 1 & 0 & 0 & -1 \\ 0 & 0 & -1 & -1 \\ 2 & 0 & 0 & 0 \end{bmatrix} \times \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \\ 0 \\ 1 \end{bmatrix}$$

The diagram illustrates the equation $A \begin{bmatrix} u_{0,0} \\ u_{1,0} \\ \vdots \\ u_{N-1,0} \\ \vdots \\ u_{N-1,N-1} \end{bmatrix} = b$. A red box on the left labeled "Shape $N^2 \times N^2$ " has an arrow pointing to the matrix A . Another red box on the right labeled "Length N^2 " has two arrows: one pointing to the vector $\begin{bmatrix} u_{0,0} \\ u_{1,0} \\ \vdots \\ u_{N-1,0} \\ \vdots \\ u_{N-1,N-1} \end{bmatrix}$ and another pointing to the vector b .

Shape $N^2 \times N^2$

Length N^2

$$A \begin{bmatrix} u_{0,0} \\ u_{1,0} \\ \vdots \\ u_{N-1,0} \\ \vdots \\ u_{N-1,N-1} \end{bmatrix} = b$$

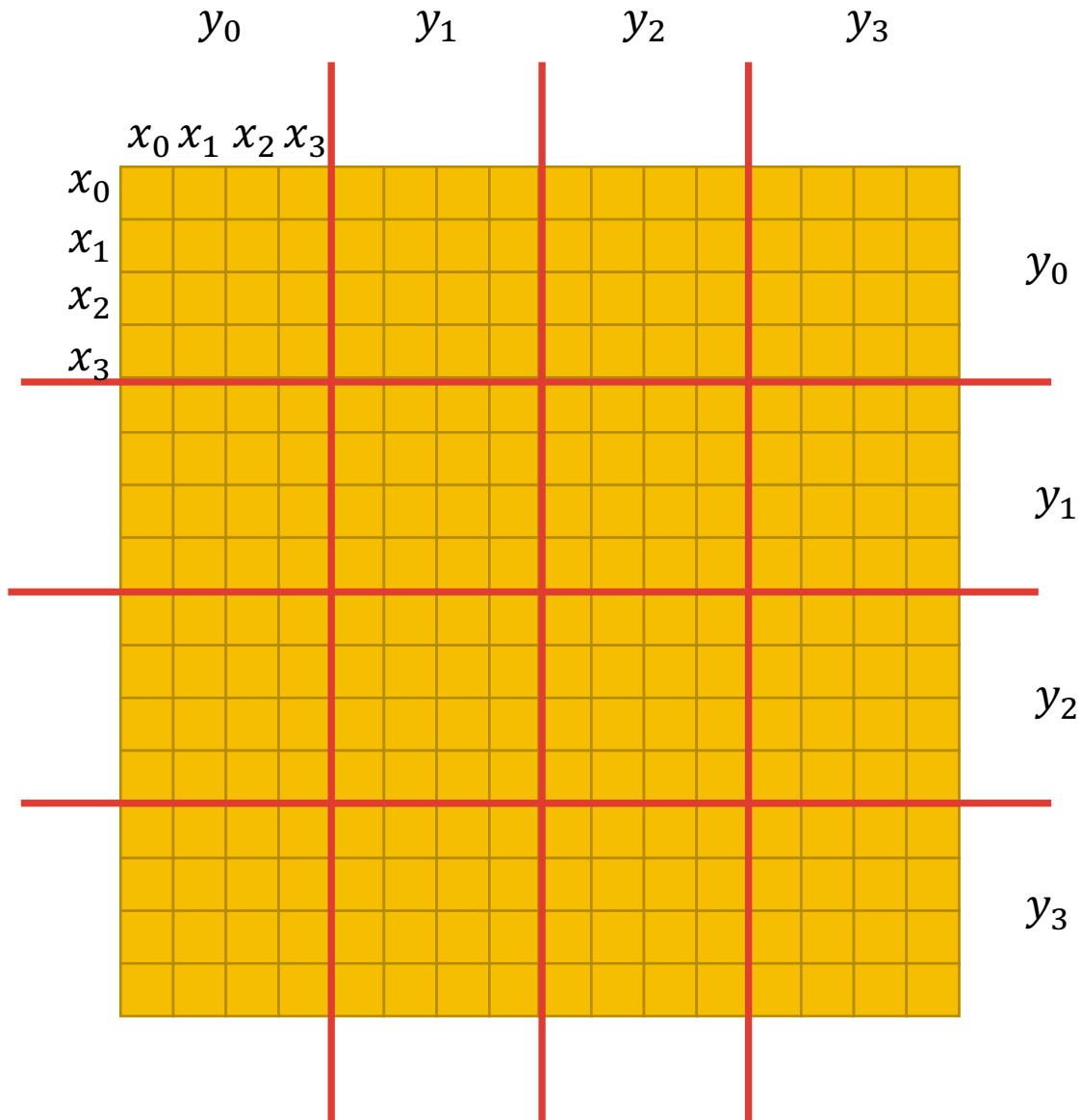
Matrix

- If $N = 100$ we would need to build a matrix of $10,000 \times 10,000$
 - ~780 MB
- If $N = 1000$ we would need to build a matrix of $1,000,000 \times 1,000,000$
 - ~7 TB
- Very quickly becomes infeasible
- Lets look at its structure.

Mapping the Matrix

$$u_{i,j} \rightarrow j * N + i$$

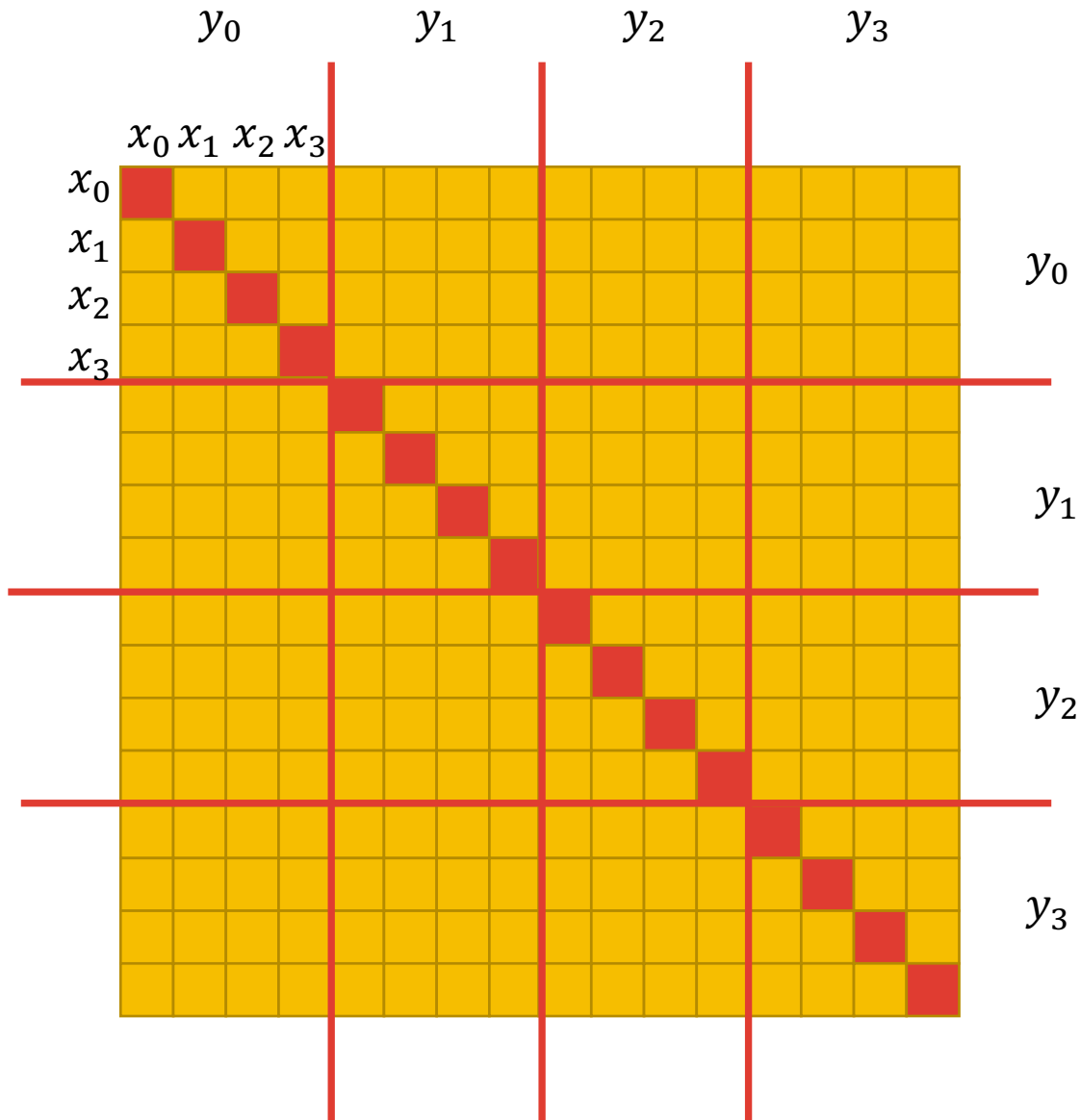
$$1u_{i,j} = 0 \text{ on boundary}$$



$$N = 4$$

$$9(4u_{i,j} - u_{i+1,j} - u_{i,j+1} - u_{i-1,j} - u_{i,j-1}) = 1$$

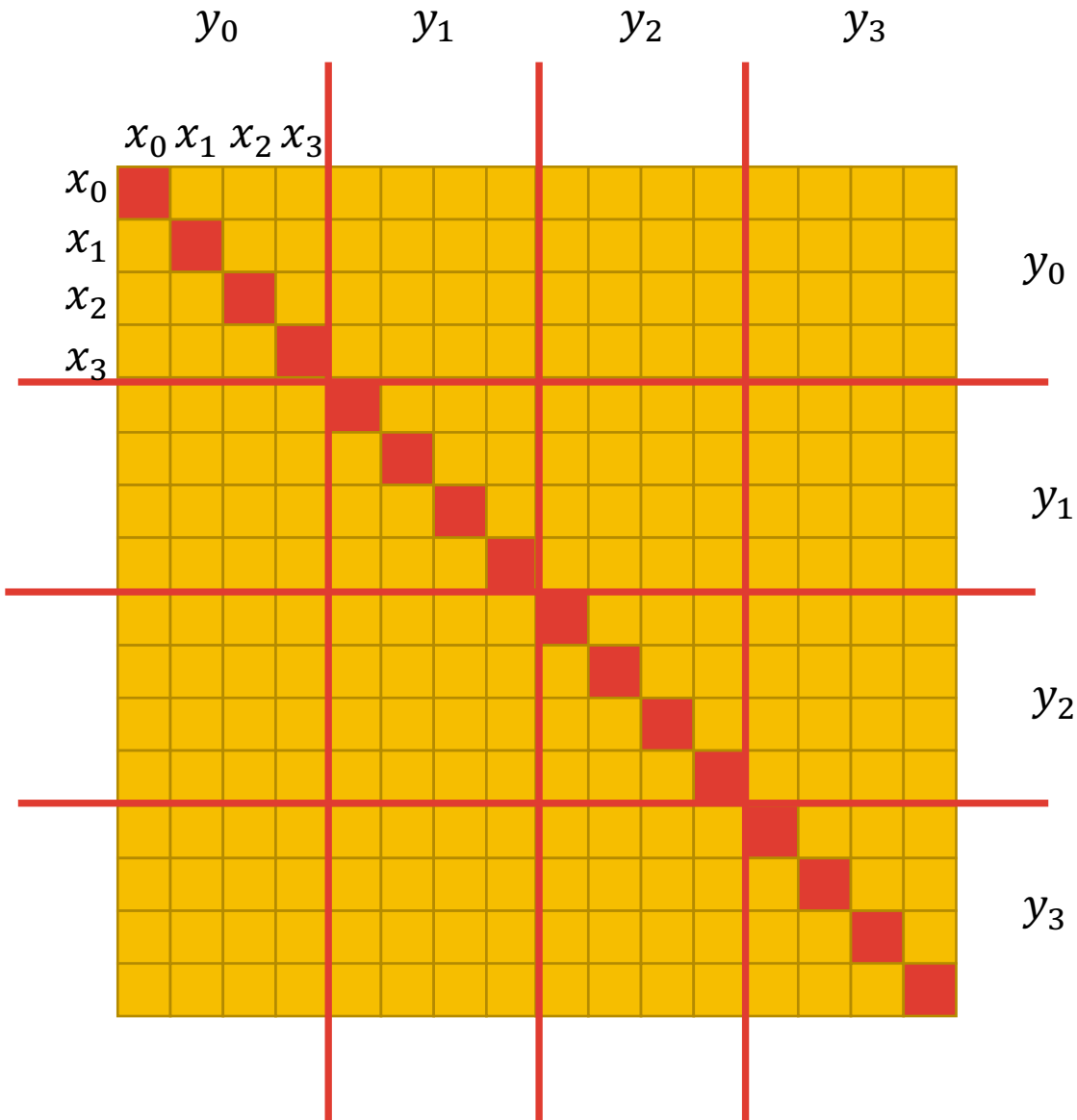
$u_{0,0}$	$u_{1,0}$	$u_{2,0}$	$u_{3,0}$
$u_{0,1}$	$u_{1,1}$	$u_{2,1}$	$u_{3,1}$
$u_{0,2}$	$u_{1,2}$	$u_{2,2}$	$u_{3,2}$
$u_{0,3}$	$u_{1,3}$	$u_{2,3}$	$u_{3,3}$



$$N = 4$$

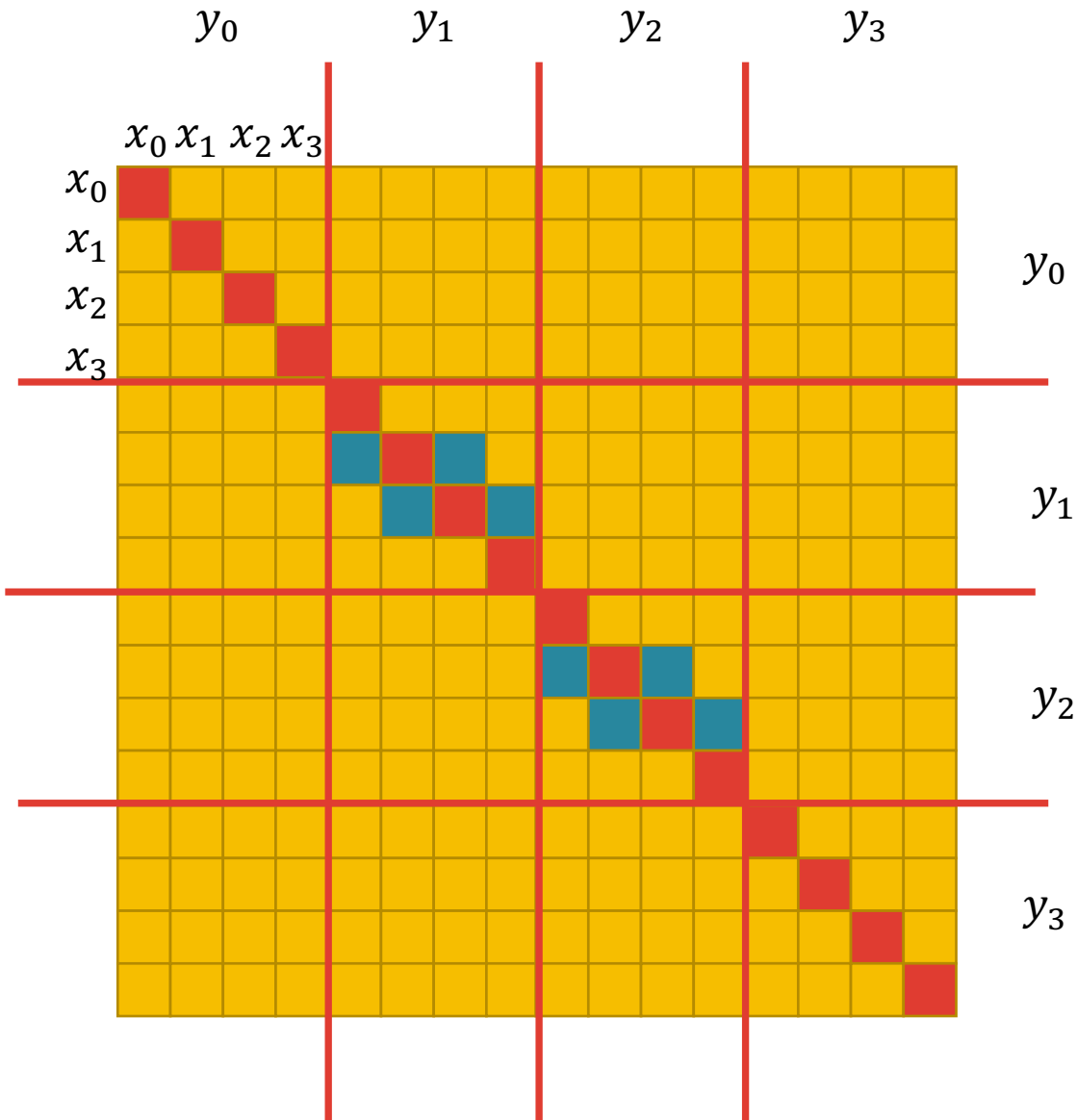
$$9(4u_{i,j} - u_{i+1,j} - u_{i,j+1} - u_{i-1,j} - u_{i,j-1}) = 1$$

$u_{0,0}$	$u_{1,0}$	$u_{2,0}$	$u_{3,0}$
$u_{0,1}$	$u_{1,1}$	$u_{2,1}$	$u_{3,1}$
$u_{0,2}$	$u_{1,2}$	$u_{2,2}$	$u_{3,2}$
$u_{0,3}$	$u_{1,3}$	$u_{2,3}$	$u_{3,3}$



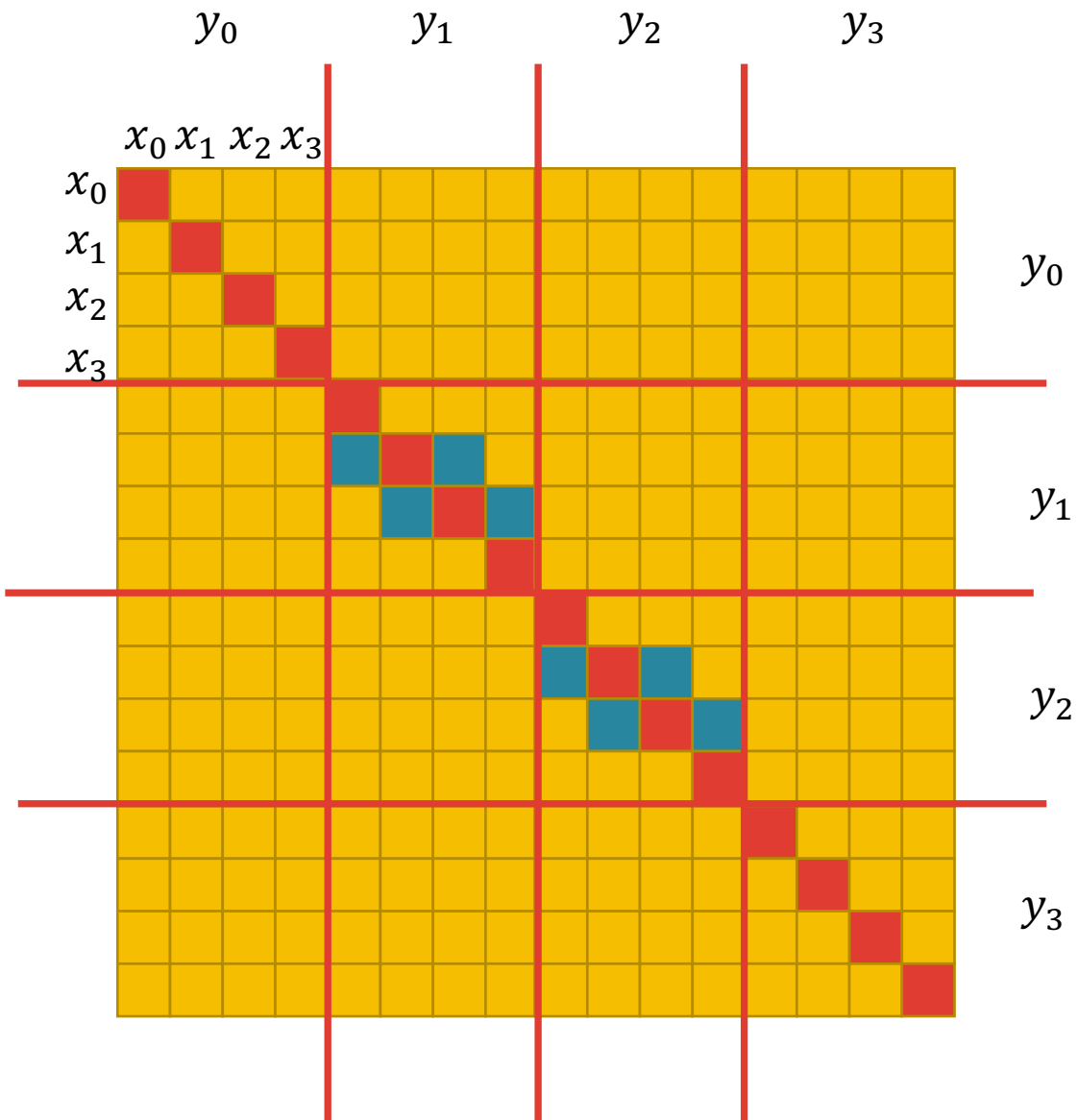
$$N = 4$$

$$9(4u_{i,j} - \underbrace{u_{i+1,j}} - u_{i,j+1} - \underbrace{u_{i-1,j}} - u_{i,j-1}) = 1$$



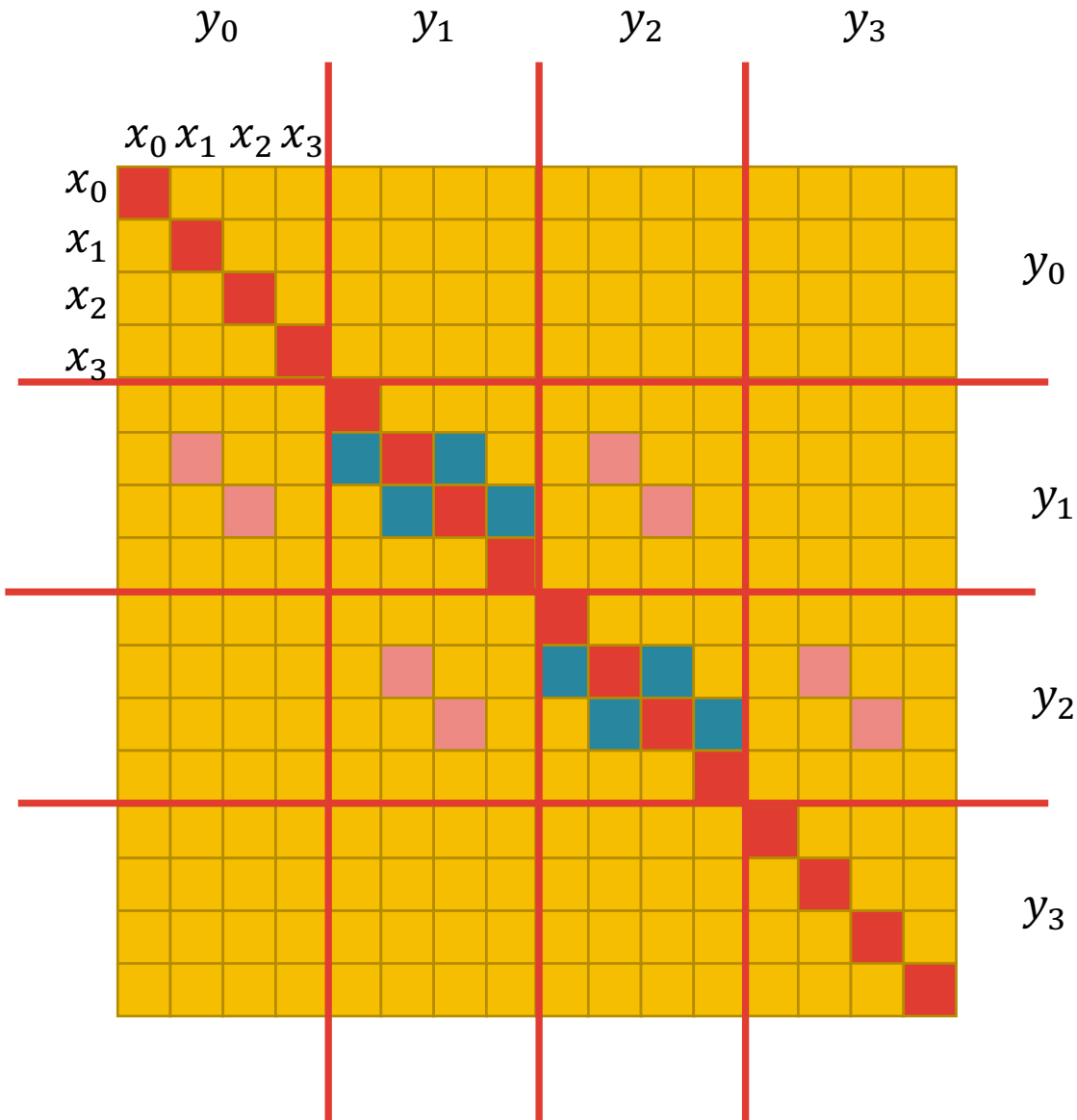
$$N = 4$$

$$9(4u_{i,j} - \underbrace{u_{i+1,j}} - u_{i,j+1} - \underbrace{u_{i-1,j}} - u_{i,j-1}) = 1$$



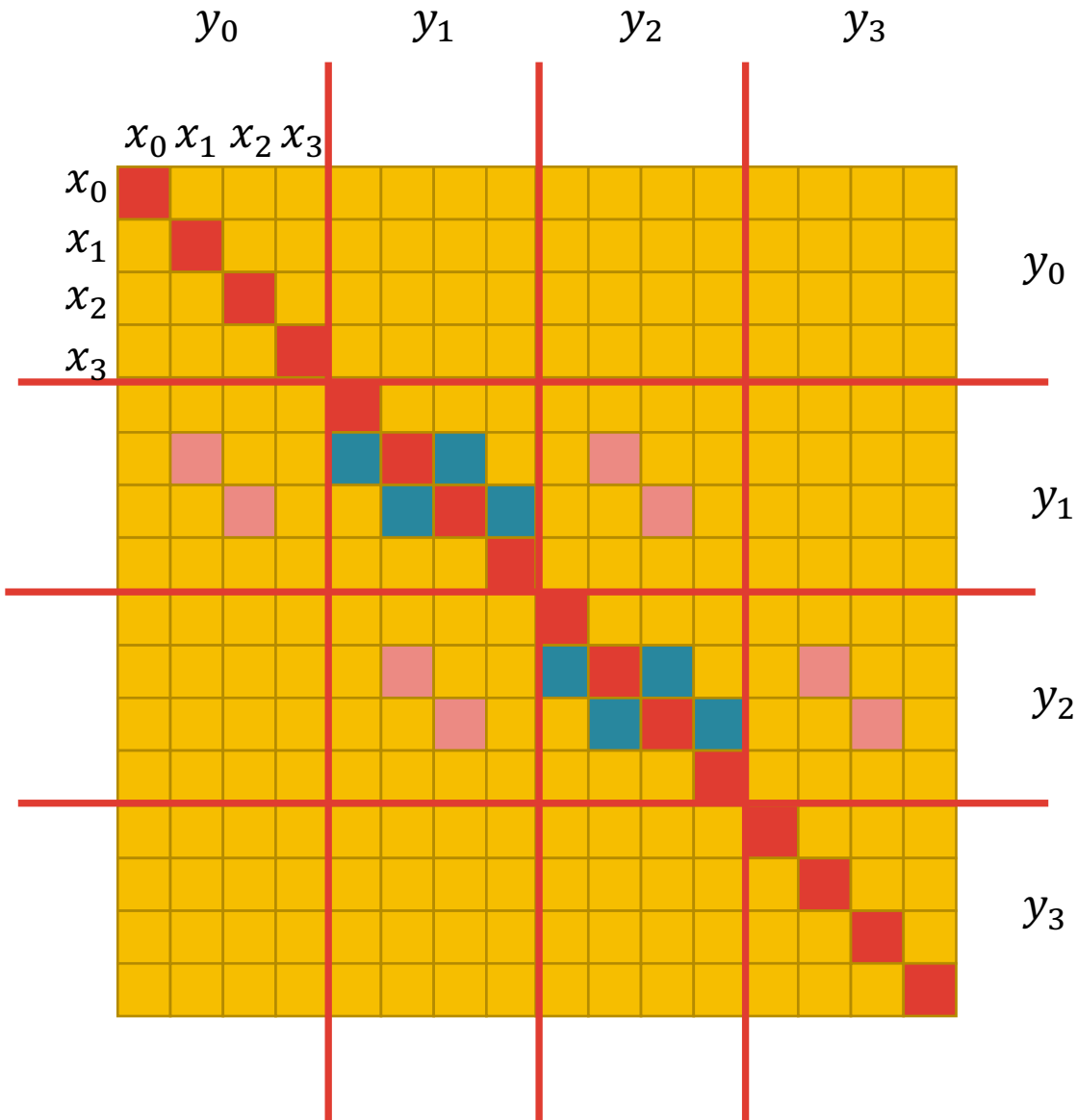
$$N = 4$$

$$9(4u_{i,j} - u_{i+1,j} - \textcircled{u_{i,j+1}} - u_{i-1,j} - \textcircled{u_{i,j-1}}) = 1$$



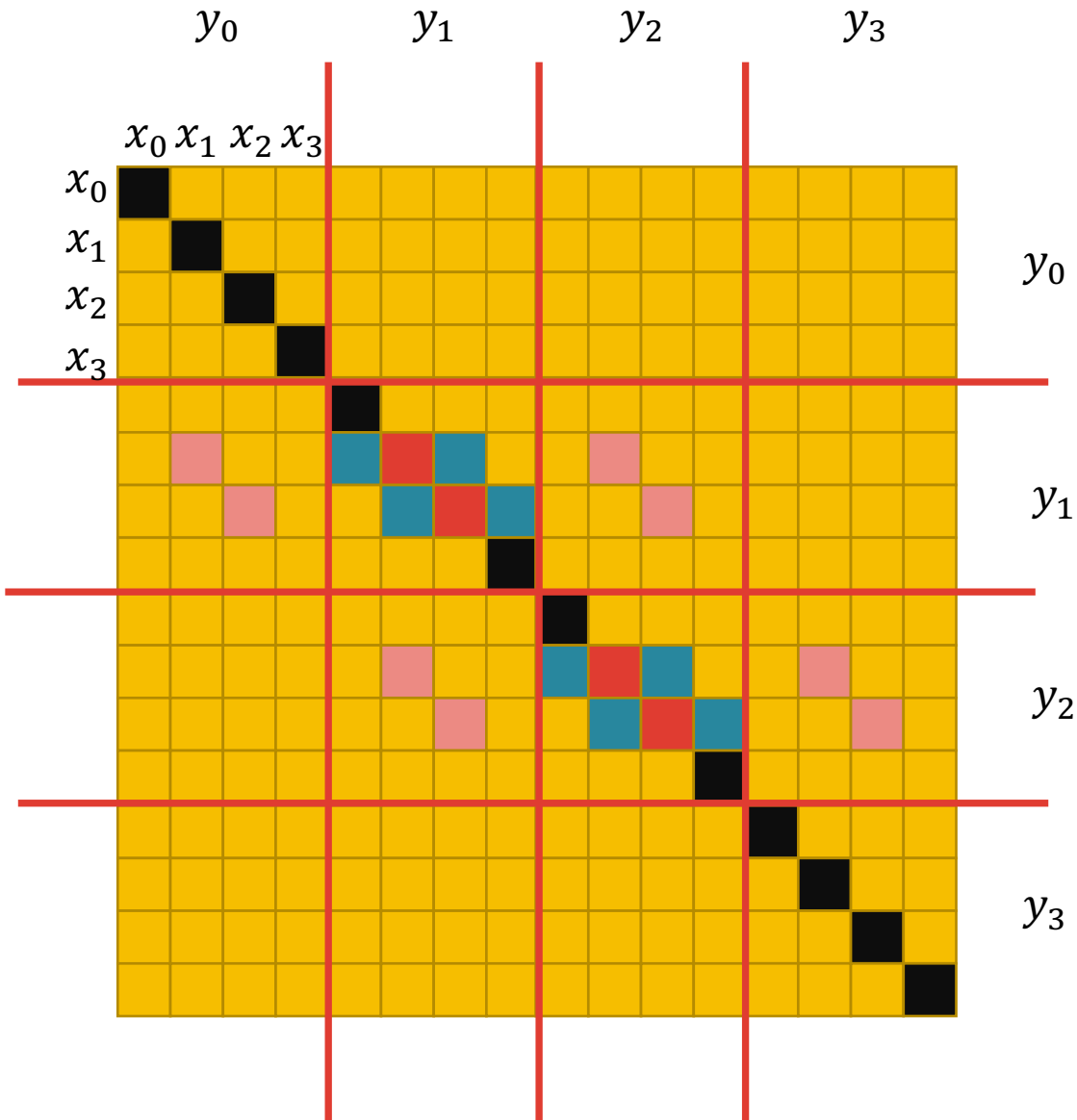
$$N = 4$$

$$9(4u_{i,j} - u_{i+1,j} - \textcircled{u_{i,j+1}} - u_{i-1,j} - \textcircled{u_{i,j-1}}) = 1$$



$$N = 4$$

$$1u_{i,j} = 0 \text{ on boundary}$$



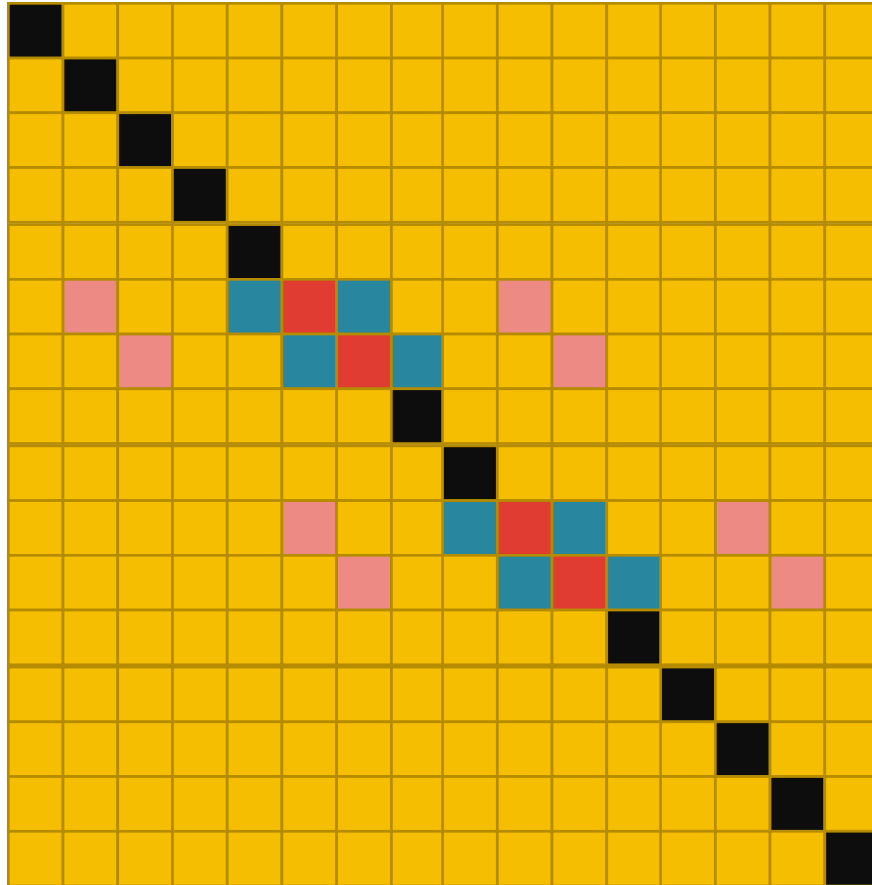
$$N = 4$$

$$1u_{i,j} = 0 \text{ on boundary}$$

$$N = 4$$

$$1u_{i,j} = 0 \text{ on boundary}$$

$$9(4u_{i,j} - u_{i+1,j} - u_{i,j+1} - u_{i-1,j} - u_{i,j-1}) = 1$$



$$\text{Red square} = 36$$

$$\text{Blue square} = -9$$

$$\text{Pink square} = -9$$

$$\text{Black square} = 1$$

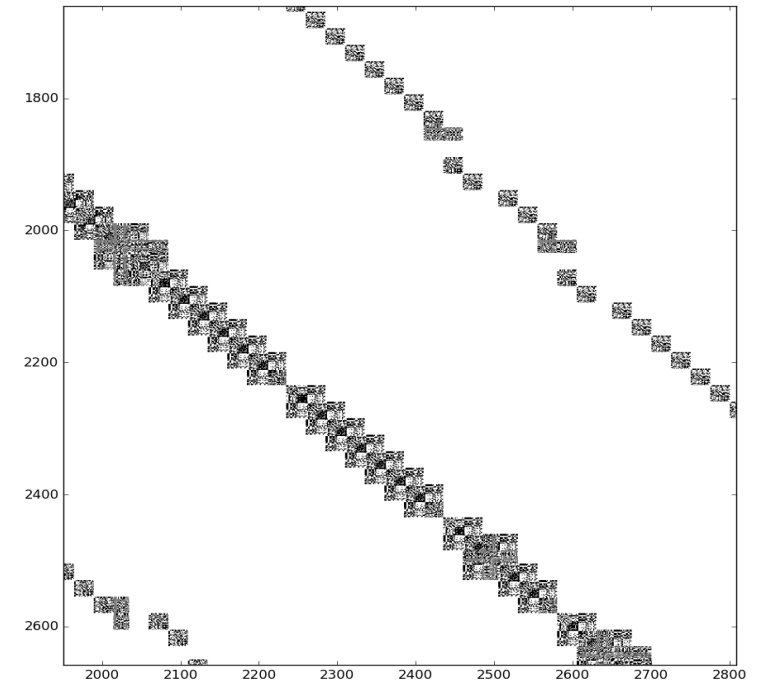
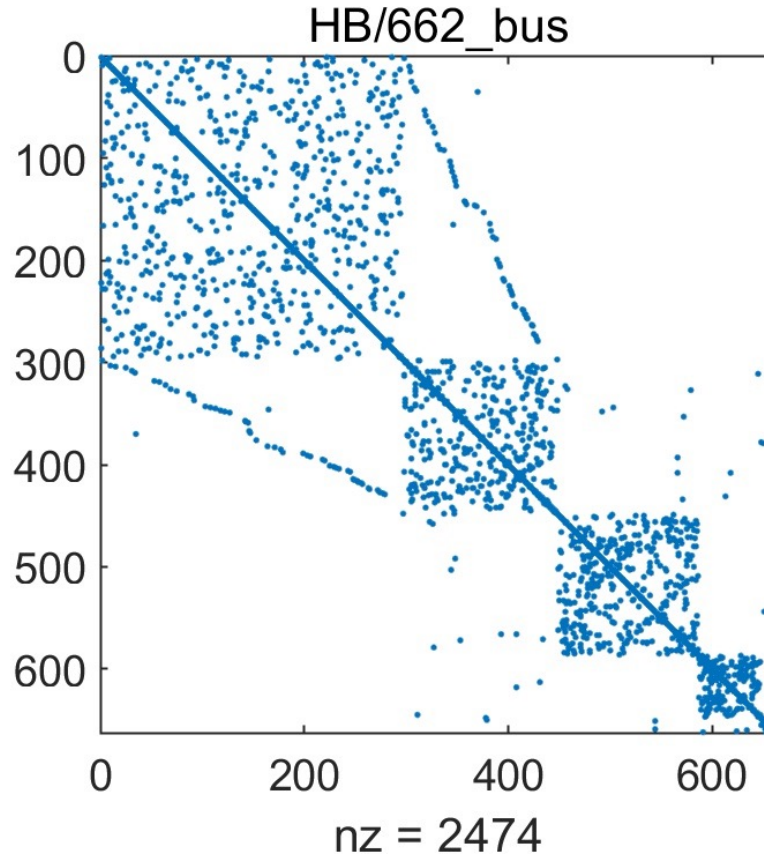
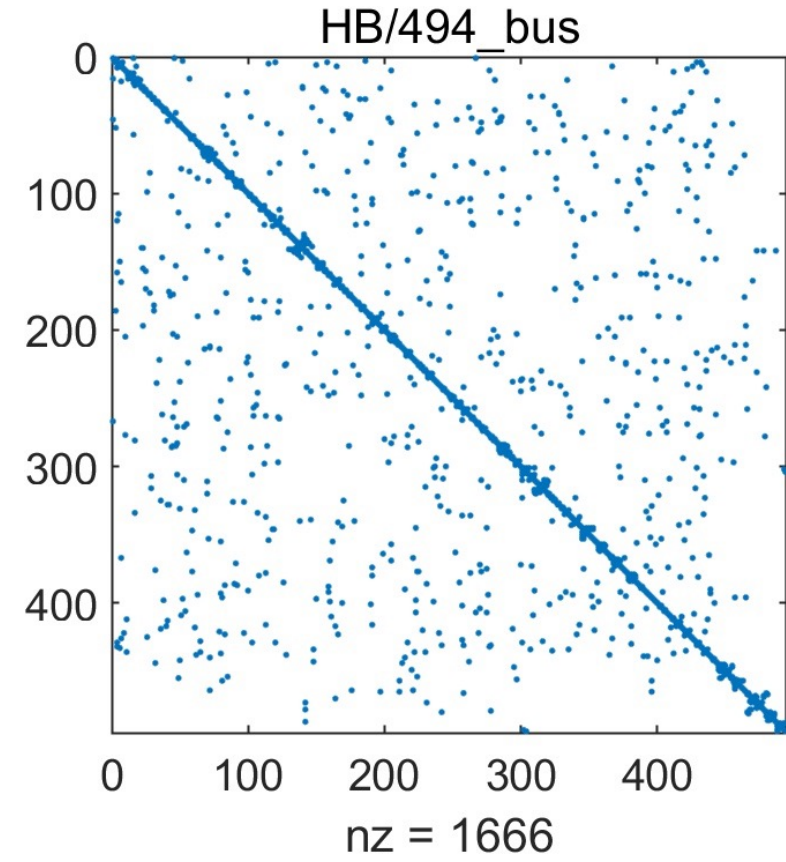
$$\text{Yellow square} = 0$$

Mostly Zero!!!

Sparse Matrix

- Matrices that are mostly zero are known as 'sparse'
- We can exploit their sparsity to store them efficiently
- We can also exploit their sparsity for performance!
- Choice of Format important to maximize performance

Sparse Matrix Types





Notebook demo



Sparse Linear Solvers

Solving Sparse Linear Systems

- Direct Solvers
 - Variants of LU decomposition
 - $O(n^3)$
 - Works well for most 2D and some 3D problems
- Iterative Methods
 - Krylov subspace methods
 - Especially great for solving eigenvalues for massive matrices.
 - Next lecture!
- Multigrid Methods
 - Apply successive discretization to solve the problem

If your sparse matrix is actually dense, then sparse versions of solvers will be much more expensive!!!